AI Code Review Platform

GenAI FastAPI Backend

This project is a backend application built using FastAPI, designed to interact with a large language model (LLM) API. I

SEVERITY	CONFIDENCE	HEALTH
90	100	66

Language	**Python**	Architecture	**Microservices**
Complexity	**high**	Sec Posture	**poor**
Bugs Found	**7**	Vulns Found	**5**
Doc Grade	**C**	Prod Ready	**not ready**

Generated May 24, 2026 at 06:33 UTC

Powered by Groq Llama 3.3 · LangGraph · ChromaDB RAG

The GenAI FastAPI Backend project exhibits a high level of complexity due to its microservices architecture, which introduces challenges in coordination and communication between components. The codebase has a fair code quality, with notable issues in bug detection, security, and documentation. The project requires significant attention to address the identified vulnerabilities, bugs, and documentation gaps to ensure its production readiness. Overall, the project's current state poses significant risks to its maintainability, security, and reliability. Immediate action is necessary to mitigate these risks and improve the project's overall health.

Key Findings

Severity	Category	Finding
CRITICAL	Security	Hardcoded API Key in the codebase
HIGH	Bugs	Uncaught Exception in genai-fastapi-backend/app/api/routes/chat.py
MEDIUM	Documentation	Lack of comprehensive docstrings and contributing guide
MEDIUM	Architecture	Complex microservices architecture with potential coordination challenges
HIGH	Security	Missing Input Validation and Insecure Deserialization

Project	GenAI FastAPI Backend
Purpose	This project is a backend application built using FastAPI, designed to interact with a large language mo
Architecture	Microservices
Languages	Python, Markdown, JSON, Text
Frameworks	FastAPI
Complexity	high
Entry Points	app/main.py, app/agents/session15/playground.py
Notes	The project uses a microservices architecture, with separate agents and tools for different tasks. This a

Codebase Strengths

✓ The project uses a microservices architecture, which allows for flexibility and scalability

✓ The project includes various tools and agents for tasks like planning, execution, and validation

✓ The project uses environment variables for configuration and a custom dependency injection system

Code Quality: **fair** · Testing Gaps: The code lacks comprehensive unit tests and integration tests, particularly for

Uncaught Exception in genai-fastapi-backend/app/api/routes/chat.py

HIGH

genai-fastapi-backend/app/api/routes/chat.py

Category: unhandled_exception

The code does not handle the exception properly, it simply raises it again without providing any useful information.

```
raise HTTPException(status_code=500, detail=f"An unexpected error occurred: {str(exc)}")
```

Fix: Provide a more informative error message and consider logging the exception for further investigation.

Potential Null Reference in genai-fastapi-backend/app/agents/session16/orchestrator.py

MEDIUM

genai-fastapi-backend/app/agents/session16/orchestrator.py

Category: null_reference

The code does not check if the 'validation' object is null before accessing its properties.

```
if not validation.approved and result.output.get("skipped")
```

Fix: Add a null check for the 'validation' object before accessing its properties.

Logic Error in genai-fastapi-backend/app/agents/session15/executor.py

MEDIUM

genai-fastapi-backend/app/agents/session15/executor.py

Category: logic_error

The code does not handle the case where the 'tool_input' is null or empty.

```
output = tool_fn(**tool_input)
```

Fix: Add a check for null or empty 'tool_input' and handle it accordingly.

Potential Type Error in genai-fastapi-backend/app/agents/session14/agent.py

LOW

genai-fastapi-backend/app/agents/session14/agent.py

Category: type_error

The code does not check the type of the 'parsed' object before accessing its properties.

```
if parsed["type"] != "error"
```

Fix: Add a type check for the 'parsed' object before accessing its properties.

Potential Race Condition in genai-fastapi-backend/app/agents/session17/policy.py

HIGH

genai-fastapi-backend/app/agents/session17/policy.py

Category: race_condition

The code uses a shared variable 'retries' without proper synchronization.

```
if retries < max_retries
```

■ **Fix:** Use a thread-safe way to update the 'retries' variable.

Anti-Pattern in genai-fastapi-backend/app/agents/session18/planner.py MEDIUM

■ genai-fastapi-backend/app/agents/session18/planner.py

Category: anti_pattern

The code uses a long chain of if-else statements to determine the plan.

```
if scenario_key not in PLAN_FACTORIES
```

■ **Fix:** Consider using a more scalable approach, such as a dictionary or a separate planning module.

Uncaught Exception in genai-fastapi-backend/app/agents/session15/validator.py HIGH

■ genai-fastapi-backend/app/agents/session15/validator.py

Category: unhandled_exception

The code does not handle the exception properly, it simply returns a ValidationResult with approved=False.

```
return ValidationResult(step_id=result.step_id, approved=False,
reason=f"Execution failed: {result.error}")
```

■ **Fix:** Provide a more informative error message and consider logging the exception for further investigation.

Critical	High	Medium	Low
1	2	1	1

Hardcoded API Key

CRITICAL

genai-fastapi-backend/WALKTHROUGH.md

A02: Cryptographic Failures

The GROQ_API_KEY is not properly secured and could be exposed.

```
GROQ_API_KEY = "gsk_..."
```

Fix: Use environment variables to store sensitive keys.

Missing Input Validation

HIGH

genai-fastapi-backend/app/agents/session14/parser.py

A03: Injection

User input is not validated, making it vulnerable to injection attacks.

```
text = raw.strip()
```

Fix: Implement input validation and sanitization.

Insecure Deserialization

HIGH

genai-fastapi-backend/app/agents/session14/parser.py

A08: Integrity Failures

The JSON parsing mechanism is vulnerable to insecure deserialization attacks.

```
parsed = _try_parse_json_response(text)
```

Fix: Use a secure deserialization mechanism.

Missing Rate Limiting

MEDIUM

genai-fastapi-backend/app/rag/playground.py

A04: Insecure Design

The application lacks rate limiting, making it vulnerable to brute-force attacks.

```
for query in sample_queries:
```

Fix: Implement rate limiting to prevent brute-force attacks.

Verbose Error Messages

LOW

genai-fastapi-backend/app/agents/session14/parser.py

A05: Security Misconfiguration

Error messages may reveal sensitive information about the application.

```
return {"type": "error", "raw": raw, "reason": validation_error}
```

■ **Fix:** Implement custom error handling to prevent information disclosure.

■■ Exposed Secrets / Credentials

• GROQ_API_KEY

Overall Grade	**C**
README Score	**70/100**
Docstring Coverage	**medium**

Quick Wins

- Add docstrings to all functions and classes
- Create a contributing guide to encourage community involvement
- Include a license file to clarify the project's licensing terms

README Improvements

Priority	Section	Suggestion
HIGH	Introduction	Add a brief overview of the project, its purpose, and its features.
HIGH	Getting Started	Provide a step-by-step guide on how to install and run the application.
MEDIUM	API Reference	Create a section that documents all available API endpoints, their parameters, and response formats.

#	Category	Effort	Impact	Action
1	Security	medium	high	Remove hardcoded API keys and implement secure key management
2	Bugs	high	high	Implement comprehensive unit tests and integration tests for error handling and edge cases
3	Documentation	low	medium	Add docstrings to all functions and classes, and create a contributing guide
4	Bugs	medium	medium	Refactor the code to address Long chain of if-else statements and Shared variables without proper synchronization
5	Security	medium	high	Implement rate limiting and input validation to prevent potential security vulnerabilities

Recommended Next Steps

1. Address the critical security vulnerabilities, including hardcoded API keys and missing input validation
2. Implement comprehensive testing, including unit tests and integration tests
3. Refactor the code to address identified bugs and anti-patterns
4. Improve documentation, including adding docstrings and creating a contributing guide
5. Conduct a thorough review of the project's architecture and identify areas for simplification and improvement

This report was generated automatically by the AI Code Review Platform using Groq Llama 3.3, LangGraph multi-agent orchestration, and ChromaDB RAG retrieval. All findings should be reviewed by a qualified engineer before remediation.